

An Analysis and Simulation of Defending Tracking Control Neural Network for Known Affine Discrete-Time Nonlinear Systems Against Adversarial Attacks

Mentee: Ablasse Kingcaid-Ouedraogo,
Mentor: Dr. Pavlo Tymoshchuk

April 30, 2026

Abstract

A neural network (NN) tracking control for discrete-time nonlinear systems with known internal dynamics is designed. The adversarial attack on the tracking control NN is modeled as additive stochastic noise injected into the output of the signal function of the controller. The adversarially perturbed system and the corresponding closed-loop tracking dynamics are derived, and the resulting degradation in tracking performance is demonstrated. The defense is based on rounding the noisy output of the signal function. When the perturbation magnitude is less than one half, the rounding recovers the correct signal function value, and the defended closed-loop dynamics reduce to the unperturbed system. The nominal C++ simulation is validated against the MATLAB reference implementation with element-wise agreement to machine precision, and the defended C++ response is compared against the nominal reference implied by the rounding-recovery condition (Appendix A).

Keywords: control system, neural network, adversarial attack, discrete-time nonlinear system, tracking control, signal function, rounding defense

Introduction

The tracking control problem is one of the most fundamental practical problems in control theory, where the goal is to design a controller that allows a system to follow a desired trajectory or reference signal over time. In recent years, neural networks have been increasingly used in control systems due to their ability to approximate complex nonlinear functions and adapt to changing conditions. However, the use of neural networks in control systems also introduces new vulnerabilities, particularly to adversarial attacks. Small, practically undetectable perturbations in the input of a neural network can lead to outsized adverse effects on the output, which can be particularly problematic in control systems where safety and reliability are paramount.

A tracking control problem can be formulated as designing such a system whose output approximately repeats a given time-varying reference trajectory [1]. The defense of a tracking control neural network against adversarial perturbations requires that, under the influence of such perturbations, tracking performance remains acceptable within certain conditions. These conditions vary and are dependent upon the designed control law's inherent robustness, the magnitude and nature of the perturbation, and the performance metrics required of the system for safe operation. The channel of attack chosen for this analysis is the output of the signal function in the tracking control system described by (6) [1].

A tracking control neural network (NN) for the known affine in the inputs, discrete-time nonlinear system was presented in [1]. The paper describes a method of tracking control that is of moderate complexity and requires no training. This method uses a signum activation function of the tracking error, as well as a diagonal matrix of the learning rate parameters to drive the system state towards the reference trajectory in finite time. The method is inherently robust to bounded uncertainties and disturbances.

This paper builds upon [1] with the following contributions:

1. An adversarial attack on the output of the signal function is modeled as additive Gaussian noise, and the resulting perturbed closed-loop dynamics are derived.
2. A defense based on rounding the noisy output of the signal function is proposed. At each time step where the noise magnitude is less than one half, the rounding recovers the correct signal function output, and the defended dynamics reduce to the unperturbed system.
3. The unperturbed simulation is validated in C++ against the MATLAB reference with element-wise agreement to machine precision; the defended C++ response is then compared against the same nominal

reference because the recovery condition reduces the defended dynamics to the unperturbed system (Appendix A).

1 A Discrete Time Tracking Control Neural Network

Discrete-time neural networks, when compared to their continuous-time counterparts, offer several advantages in control applications. They are often easier to implement in digital hardware, can be more computationally efficient, and can be designed to operate at specific sampling rates. Another distinct advantage is their relative resilience to perturbations [2]. They can also be efficiently implemented and miniaturized with digital hardware. Using Euler approximation, we consider a known affine discrete-time nonlinear system described by the following difference equation [3]

$$x(k+1) = x(k) + \tau[f(x(k)) + g(x(k))u(k)], \quad x(1) = x_1, \quad (1)$$

where $x(k) \in \mathfrak{R}^n$ is the discrete-time state variable, $f(x(k)) \in \mathfrak{R}^n$ is the nonlinear function of system internal dynamics, $g(x(k)) \in \mathfrak{R}^{n \times m}$ is the nonlinear function of system coupling dynamics, $u(k) \in \mathfrak{R}^m$ is the control input applied to the system, and $\tau = t(k+1) - t(k)$ is the discrete-time step, k . We suppose that $f(0) = 0$ and the system is stabilizable on a set $\psi \in \mathfrak{R}^m$ that includes the origin, i.e., there exists such a function $u(k) \in \mathcal{U}$ of discrete-time control that the system is asymptotically stabilized on ψ . The functions of system internal dynamics $f(x(k))$ and $g(x(k))$ are presumably known for any $k \in [1, k_{max}]$, $k_{max} < \infty$. We also suppose that the system is observable and controllable, i.e., $g(x(k)) \neq 0$. Otherwise, if $g(x(k)) = 0$, the system is uncontrolled and can be given by

$$x(k+1) = x(k) + \tau f(x(k)), \quad x(1) = x_1. \quad (2)$$

The real-valued cost function associated with (1) can be presented as follows [1]:

$$C(x) = \sum_{k=1}^{\infty} \Phi(e(k), u(k)), \quad (3)$$

where $e(k) = x(k) - r(k)$ is the tracking error, $r(k) \in \mathfrak{R}^n$ is the reference trajectory, and $\Phi(e, u) \in \mathfrak{R}$ is the positive definite function. The tracking control problem is to determine a control $u(k)$ such that, starting from any finite initial state $-\infty < x(1) < \infty$, as $k \rightarrow \infty$, (3) is minimized, i.e., the optimal value function

$$C^*(x) = \min_u \sum_{k=1}^{\infty} \Phi(e(k), u(k)), \quad (4)$$

while the state $x(k)$ remains bounded. Equation (4) means that starting from any initial value, (3) should converge to its minimum, i.e., the system should be stable. We design the cost function (3) as $C(x) = \|e\|$. Since the cost function is convex, it has a unique global minimum. Its minimizer $C^* = 0$ can be found using a gradient descent method [2].

The control $u(k)$ can be determined from (1) considering (4) by the following control equation [1]:

$$u(k) = \begin{cases} g(x(k))^{-1}(-f(x(k)) + \Delta r(k) - \beta \operatorname{sgn}(x(k) - r(k))) & \text{if } g(x(k)) \neq 0 \\ 0 & \text{if } g(x(k)) = 0 \end{cases}, \quad (5)$$

where $\beta = \operatorname{diag}(\beta_1, \dots, \beta_n)$ is the diagonal matrix of learning rate parameters and $\Delta r(k) = r(k+1) - r(k)$. Given a reference trajectory $r(k)$, the system (1) seeks to minimize the closed-loop tracking error, described by the associated difference equation with variable structure [1]

$$e(k+1) = e(k) - \beta \operatorname{sgn}(e(k)), \quad (6)$$

where

$$\operatorname{sgn}(e(k)) = \begin{cases} 1 & \text{if } e(k) > 0 \\ 0 & \text{if } e(k) = 0 \\ -1 & \text{if } e(k) < 0 \end{cases}$$

is the signum activation function [2] of tracking error $e(k) = x(k) - r(k)$. Notice that the difference equation (6) includes the signum activation function and describes supervised learning with learning rate parameter β that can be used for changing the convergence rate of the state variable trajectory $x(k)$ to the reference trajectory $r(k)$. Since the activation function and supervised learning are the basic characteristics of artificial NNs [2], we treat the discrete-time tracking controller described by the tracking equation (7) and control equation (5) as a tracking control NN. The difference equation (1) and control equation (5) describe the full closed-loop

dynamics of the nonlinear system. Substituting $u(k)$ from (5) into (1) gives the following tracking equation [1] that describes the discrete-time state variable trajectory of the designed system:

$$x(k+1) = x(k) - \beta \operatorname{sgn}(x(k) - r(k)) + \Delta r(k). \quad (7)$$

As one can see in (7), $x(k)$ and $\Delta x(k) = x(k+1) - x(k)$ can be derived by solving the difference equation for each time step $k, k = 1, 2, \dots$. Based on determined $x(k)$ and $\Delta x(k)$, the control $u(k)$ can be obtained using (5) by solving the system of linear algebraic equations. The desired discrete-time state variable trajectory $x(k)$ of the controlled system is then found by numerical integration of the difference equation (1) using known values of $x(1) = x_1, f(x), g(x)$, and $u(k)$.

2 Adversarial Attacks

The projected gradient descent (PGD) method of defense of neural networks against adversarial attacks is described in [4]. In this paper, we examine an alternative method of attack modeling using additive stochastic disturbance [5] on the output of the signal function of the controller. The adversarially perturbed system, considering (1), can be described by

$$\tilde{x}(k+1) = x(k) + \tau [f(x(k)) + g(x(k))\tilde{u}(k)], \quad (8)$$

where

$$\begin{aligned} \tilde{u}(k) &= g(x(k))^{-1}(-f(x(k)) + \Delta r(k) - \beta(\operatorname{sgn}(e(k)) + \epsilon(k))), \\ e(k) &= x(k) - r(k), \end{aligned}$$

and $\epsilon(k)$ is a random noise with normal distribution, zero mean and standard deviation σ . The adversarial disturbance $\epsilon(k)$ is modeled as an additive stochastic perturbation on the output of the signal function. This disturbance model follows the treatment of controller nonlinearities in [3], where random noises with normal distribution and zero mean are shown to affect the output of the activation function and nonlinear dynamics of the controlled system.

Substituting the control function from (5) into the perturbed system model gives the adversarially perturbed closed-loop state equation

$$\tilde{x}(k+1) = x(k) - \beta(\operatorname{sgn}(e(k)) + \epsilon(k)) + \Delta r(k). \quad (9)$$

3 Defense by Rounding of the Signal Function

The adversarial attacks discussed in this paper degrade tracking performance by perturbing the output of the signal function used in the controller. When additive noise is injected into the output of the signal function, the defense is based on rounding the noisy output to the nearest integer value.

Consider the signal function $\operatorname{sgn}(e(k))$ from (6) subject to additive noise $\epsilon(k)$. The noisy signal function output is

$$\tilde{s}(k) = \operatorname{sgn}(e(k)) + \epsilon(k), \quad (10)$$

where $\epsilon(k)$ is a stochastic perturbation. Since the clean output of sgn takes values in $\{-1, 0, +1\}$, the correct value can be recovered by rounding, provided $|\epsilon(k)| < \frac{1}{2}$. The defended output of the signal function is defined by

$$\hat{s}(k) = \begin{cases} +1 & \text{if } \tilde{s}(k) > +\frac{1}{2}, \\ 0 & \text{if } |\tilde{s}(k)| \leq \frac{1}{2}, \\ -1 & \text{if } \tilde{s}(k) < -\frac{1}{2}. \end{cases} \quad (11)$$

When $|\epsilon(k)| < \frac{1}{2}$, the rounding operation satisfies $\hat{s}(k) = \operatorname{sgn}(e(k))$, so that the defended control input

$$\hat{u}(k) = g(x(k))^{-1}(-f(x(k)) + \Delta r(k) - \beta\hat{s}(k)) \quad (12)$$

is identical to the unperturbed control input (5), and the defended closed-loop state equation

$$x(k+1) = x(k) - \beta\hat{s}(k) + \Delta r(k) \quad (13)$$

reduces to the unperturbed system (7).

The functional block-diagram of the defended system described by (10)–(13) is presented in Fig. 1. The tracking error $e(k)$ enters the signal function block sgn , whose output is subject to the additive stochastic perturbation $\epsilon(k)$, producing the noisy signal $\tilde{s}(k)$ from (10). The noisy signal is rounded to the defended value $\hat{s}(k)$ according to (11). The defended output is then passed to the amplification block β . The amplified signal, together with the reference increment Δr and the state-dependent drift $f(x(k))$, enters the LAESS block, which combines these with $g(x(k))$ to compute the defended control input $\hat{u}(k)$.

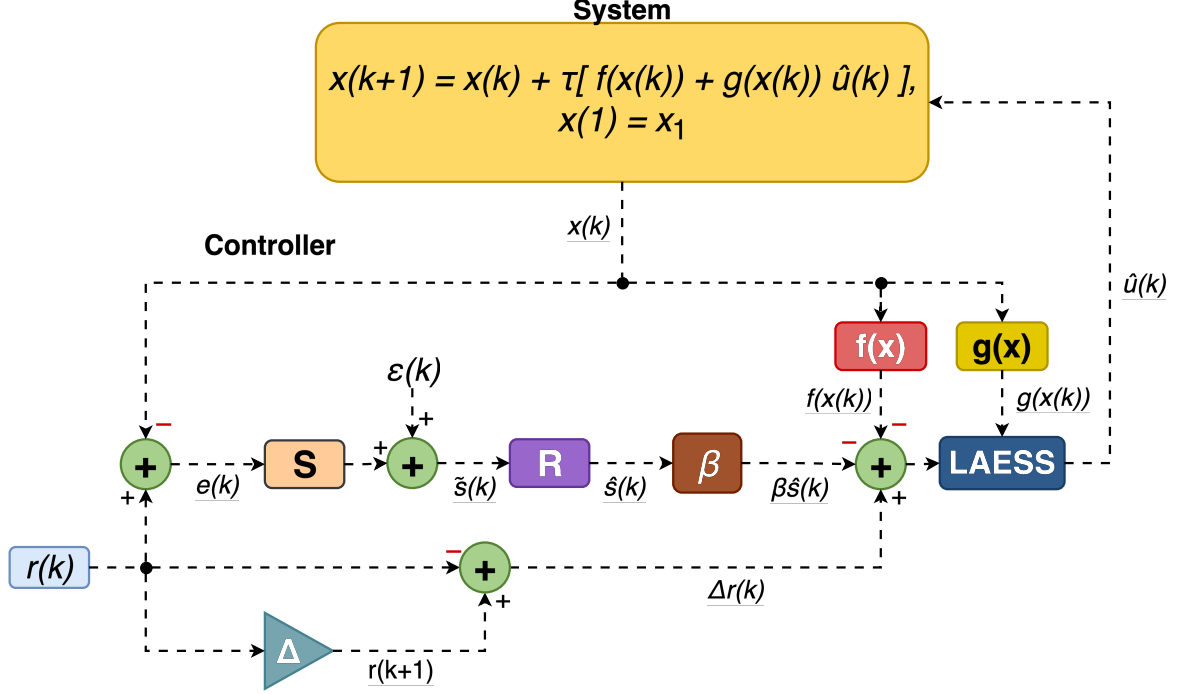


Figure 1: Functional block-diagram of the discrete-time controlled system with rounding defense described by (11)–(13).

4 Simulations

4.1 Simulation of Unperturbed System

A Continuous Stirred-Tank Reactor (CSTR) is a chemical engineering system in which reactants are continuously fed into a well-mixed vessel and products are continuously withdrawn, maintaining a homogeneous composition throughout the reactor volume. The CSTR is a standard benchmark for nonlinear control because its dynamics exhibit strong nonlinearity and coupling between states.

The operation of the discrete-time controlled nonlinear system described by (1) and control equation (5) using a nonlinear cooling CSTR adopted from [6, 7, 8] was simulated in [1]. The dynamics of the reactor can be described in normalized form by (1), where

$$\mathbf{f} = \begin{bmatrix} x_1(k) + \tau \left(-\alpha x_1(k) + D(1 - x_1(k))e^{\frac{x_2(k)}{1 + \frac{x_2(k)}{\gamma}}} \right) \\ x_2(k) + \tau \left(-\alpha x_2(k) + BD(1 - x_1(k))e^{\frac{x_2(k)}{1 + \frac{x_2(k)}{\gamma}}} - \lambda x_2(k) \right) \end{bmatrix}, \quad \mathbf{g} = \begin{bmatrix} 0 \\ \lambda \end{bmatrix}, \quad (14)$$

with $\alpha = 1.0$, $\lambda = 0.3$, $\gamma = 20.0$, $B = 1.0$, $D = 0.072$, $\beta = 100$, $\tau = 0.001$ s, and $x_1(1) = x_2(1) = 0$. Here, x_1 and x_2 denote the dimensionless reactant concentration and reactor temperature, respectively; α is the dilution rate; D is the Damköhler number; γ is the dimensionless activation energy; B is the dimensionless adiabatic temperature rise; and λ is the dimensionless heat transfer coefficient. The learning rate β is defined in (5), and τ is the discretization time step.

The purpose is to track the reference trajectories

$$r_1(t) = \begin{cases} 0.4472, & \text{if } 0 \leq t \leq 15 \\ 0.7646, & \text{if } 15 < t < 30 \\ 0.4472, & \text{if } 30 \leq t < 45 \end{cases} \quad \text{and} \quad r_2(t) = \begin{cases} 2.7520, & \text{if } 0 \leq t \leq 15 \\ 4.7052, & \text{if } 15 < t < 30 \\ 2.7520, & \text{if } 30 \leq t < 45. \end{cases} \quad (15)$$

The trajectory of state variable $x_2(t)$ obtained based on (1) and control equation (5) and the reference trajectory $r_2(t)$ are presented in Fig. 2 (top panel). The tracking error $e_2(t) = x_2(t) - r_2(t)$ is shown in the bottom panel, confirming the finite-time convergence property of the control law (6): the error spikes at each reference transition and decays to zero between transitions.

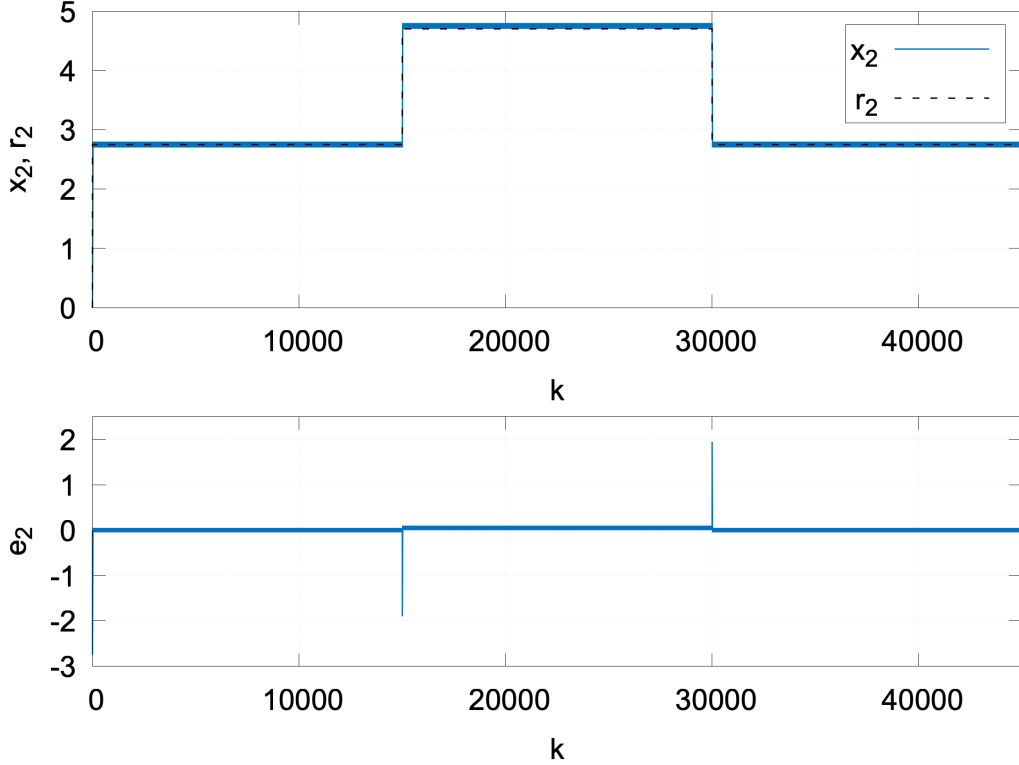


Figure 2: Unperturbed CSTR simulation. Top: state $x_2(t)$ (solid blue) vs reference $r_2(t)$ (dashed black). Bottom: tracking error $e_2(t) = x_2(t) - r_2(t)$ over the full trajectory.

4.2 Simulation of Attacked System

The CSTR system from (14) is simulated under the perturbed closed-loop dynamics (9). At each time step, the additive disturbance $\epsilon(k)$ is drawn from a zero-mean Gaussian source with standard deviation $\sigma = 1$ and rejection-sampled to satisfy $|\epsilon(k)| < \frac{1}{2}$, so the perturbation is bounded by the rounding-recovery threshold in every realization.

The attacked CSTR exhibits degraded tracking with visible oscillations of $x_2(t)$ around the reference trajectory $r_2(t)$. A direct comparison against the defended system is shown in Fig. 3.

4.3 Simulation of Defended System

The rounding defense (11) is applied to the CSTR system from (14) using the same rejection-sampled disturbance as the attacked simulation, so $|\epsilon(k)| < \frac{1}{2}$ holds by construction at every time step. The sufficient condition of the rounding defense is therefore satisfied at every k , and $\hat{s}(k) = \text{sgn}(e(k))$ exactly. The defended closed-loop state equation (13) consequently reduces to the unperturbed closed-loop system (7) at every step, and the defended trajectory coincides with the unperturbed response over the full simulation.

Fig. 3 presents the defended state trajectory $x_2(t)$ overlaid with the attacked trajectory for direct comparison (top panel), along with tracking error $e_2(t)$ for both systems (bottom panel). The defended trace reproduces the unperturbed reference response, while the attacked trace exhibits the degraded tracking of the undefended system.

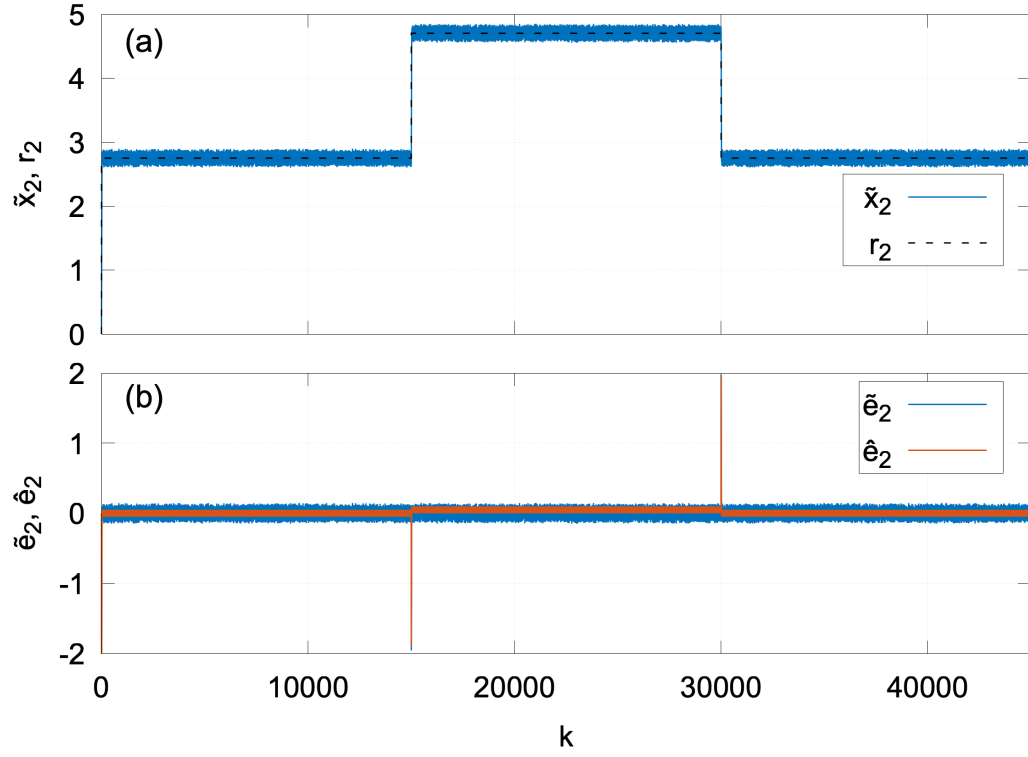


Figure 3: Comparison of attacked and defended CSTR trajectories. Top: state x_2 under attack (solid blue) and under rounding defense (solid orange) against reference r_2 (dashed black). Bottom: tracking error $e_2(t) = x_2(t) - r_2(t)$ under attack (blue) vs defense (orange) over the full trajectory.

References

- [1] P. Tymoshchuk, “A tracking control neural network for the known affine in the inputs discrete-time nonlinear systems,” University of North Texas, Tech. Rep., 2024.
- [2] A. Cichocki and R. Unbehauen, *Neural Networks for Optimization and Signal Processing*. New York, NY, USA: Wiley, 1993.
- [3] P. Tymoshchuk, “Neural network optimal control for discrete-time nonlinear systems with known internal dynamics,” *Neural Computing and Applications*, vol. 36, pp. 4421–4434, Mar. 2024.
- [4] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, “Towards deep learning models resistant to adversarial attacks,” 2019. [Online]. Available: <https://arxiv.org/abs/1706.06083>
- [5] H. Hajri, M. Césaire, L. Schott, S. Lamprier, and P. Gallinari, “Neural adversarial attacks with random noises,” *International Journal on Artificial Intelligence Tools*, vol. 32, no. 6, p. 2360010, aug 2023.
- [6] Z. Yan and J. Wang, “Model predictive control of nonlinear affine systems based on the general projection neural network and its application to a continuous stirred tank reactor,” in *Proc. Intern. Conf. Information Science and Technology*, 2011, pp. 1011–1015.
- [7] A. Soukkou, A. Khellaf, S. Leulmi, and K. Boudeghdegh, “Optimal control of a CSTR process,” *Brazilian Journal of Chemical Engineering*, vol. 25, no. 4, pp. 799–812, 2008.
- [8] C.-F. Juang and I.-F. Chung, “Recurrent fuzzy network design using hybrid evolutionary learning algorithms,” *Neurocomputing*, vol. 70, no. 16–18, pp. 3001–3010, 2007.

A C++ Implementation and Performance Comparison

To verify the simulation results and assess computational efficiency, the MATLAB reference implementation was ported to C++. The port uses Eigen matrices for a column-major $2 \times N$ data layout (row 0: state x_1 , row 1: state x_2), pre-computes the reference difference matrix ΔR before the simulation loop, and replaces MATLAB’s conditional signum with a branchless arithmetic template. Gnuplot replaces MATLAB’s native plotting for publication-quality figures.

A.1 Output Comparison

The C++ implementation produces state trajectories, control inputs, and tracking errors that are visually indistinguishable from the MATLAB reference. Element-wise comparison (see Numerical Validation below) confirms that the unperturbed outputs agree to machine precision. For the attacked simulation, because each run draws independent noise samples, exact numerical equivalence is not expected; however, the qualitative degradation pattern is consistent across both implementations. Since the graphs are the same, only the C++ outputs are shown in the simulation sections of the main text.

A.2 Numerical Validation

Element-wise comparison of the unperturbed outputs yields a maximum deviation on the order of 10^{-15} across all state variables, consistent with IEEE 754 double-precision machine epsilon of 2.22×10^{-16} . The two implementations produce identical floating-point results up to rounding in the final digits.

A.3 Runtime Performance

Table 1: Runtime comparison between MATLAB and C++ CSTR simulations ($n = 45,000$ steps, $\tau = 0.001$ s).

Simulation	MATLAB (s)	C++ (s)
Unperturbed	1.08	4.11×10^{-3}
Perturbed	1.16	4.64×10^{-3}

The runtime speedup is computed as the MATLAB elapsed time divided by the C++ elapsed time for the same 45,000-step CSTR simulation workload. The resulting ratios show that the C++ implementation reduces elapsed simulation time by approximately two orders of magnitude in both the unperturbed and perturbed configurations.